

BEARCAT CORPORATION

Solution Design Document

Employee Onboarding Process Automation

1. Purpose and Scope

1.1 Document Purpose

This Solution Design Document (SDD) defines the technical architecture, component design, integration specifications, test cases, and security controls for automating the Employee Onboarding process at Bearcat Corporation using UiPath Studio Web, a Flask REST API hosted on Render, and Google Sheets.

1.2 In-Scope

- Automated reading from the Google Sheets new hire tracking file
- Risk Policy classification via the Risk Control Policy PDF (RCP-HR-2025-001), read and served by the Flask API
- Background check status retrieval and routing logic via a simulated vendor REST API (Flask)
- Conditional evaluation of all onboarding prerequisites per RCP-HR-2025-001
- IT provisioning notification via the built-in Flask email inbox system
- Exception handling: Try-Catch wrapping all row processing logic
- Automated write-back of all processing results to Google Sheets (columns I through AA)

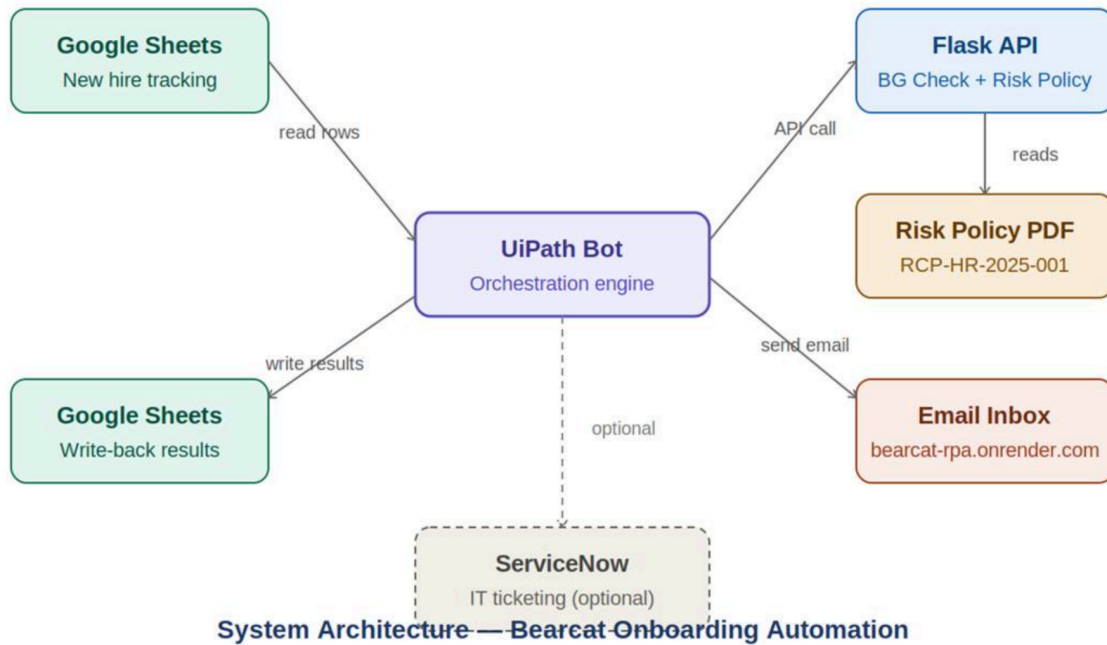
1.3 Out of Scope

- Employee termination, transfer, international onboarding, or offboarding
- Direct deposit configuration, benefit enrollment, payroll, and W-2 setup
- Rehire cases with pre-existing employee records and training assignment integration
- Email delivery via external SMTP service (Mailtrap replaced by built-in Flask inbox due to rate limits)
- Azure Active Directory provisioning (replaced by IT provisioning email for demo)
- ServiceNow ticket creation (replaced by bot-generated IT Ticket ID for demo)
- Microsoft Outlook / Office 365 SMTP integration (Microsoft security policies block automated SMTP authentication from unrecognized applications on personal accounts; replaced by Flask inbox)

2. Solution Architecture

2.1 System Architecture Overview

The bot is orchestrated by UiPath Studio Web. It reads new hire data from Google Sheets, calls the Flask REST API to retrieve risk policy classification and background check status, evaluates all conditions, sends emails via the Flask inbox, and writes all results back to Google Sheets. The Flask API is the single integration point for both the Risk Control Policy PDF and the simulated background check vendor.



2.2 Technology Stack

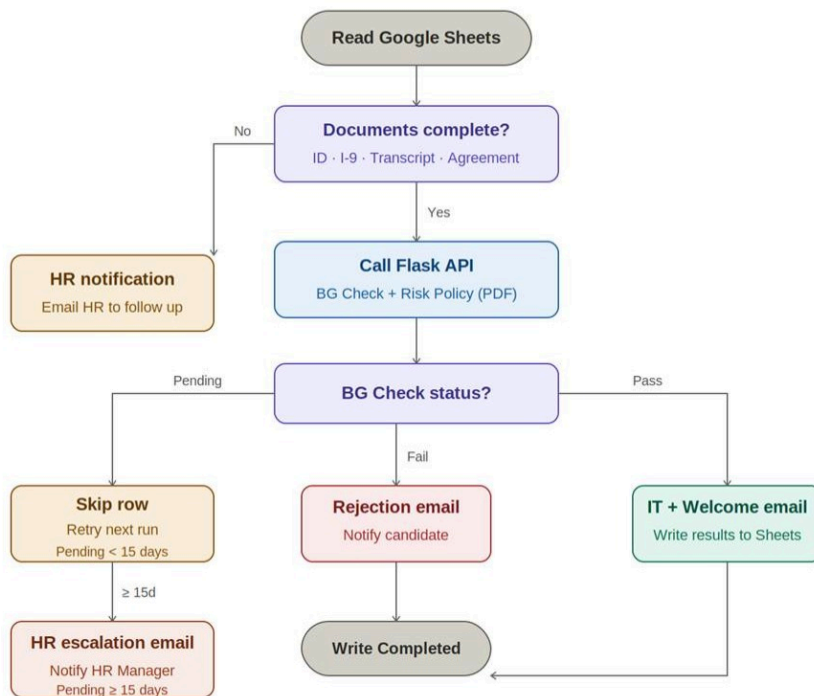
Automation Platform	UiPath Studio Web
Data Sources	Google Sheets + Risk Control Policy PDF (RCP-HR-2025-001) NewHireTracking_Bearcat - Google Sheets bearcat-RPA/RiskControlPolicy_Bearcat.pdf at main · xtzx402/bearcat-RPA
Flask API (Render)	Background check simulation + Risk policy engine + Email inbox Github repo link: https://github.com/xtzx402/bearcat-RPA Email inbox link: https://bearcat-rpa.onrender.com/inbox
Version	2.0
Date Updated	May 3, 2026
Policy Reference	RCP-HR-2025-001 Version 2.2
Status	In Review

Layer	Technology	Purpose
Orchestration	UiPath Studio Web	Main workflow engine; coordinates all steps and decision logic
Data Input/Output	Google Sheets	New hire tracking file; trigger source and write-back log (cols I–AA)
Policy & BG Check	Flask REST API (Render)	Reads Risk Control Policy PDF; returns risk classification + BG result
Policy Document	PDF RCP-HR-2025-001	Authoritative compliance reference; rules encoded in Flask API
Email Inbox	Built-in Flask Inbox	Receives all bot emails; viewable at /inbox — no SMTP required
Write-Back	Google Sheets Write Cell	Bot writes all results after each row is processed

2.3 Data Flow Summary

1. UiPath reads Google Sheets → 2. Calls Flask API with employee data → 3. Flask reads PDF, returns risk + BG result → 4. UiPath evaluates routing logic → 5. Branch-specific emails sent via Flask /send-email → 6. All results written back to Google Sheets columns I–AA.

3. Process Flow



Process Decision Flow — Bearcat Onboarding Bot

3.1 Routing Logic

BG Status	Days Waiting	Bot Action
Pass	Any	Send IT provisioning email + Welcome email; write all results to Sheets
Fail	Any	Send rejection email to candidate; write Completed + notes to Sheets
Pending	< 15 days	Skip row; retry on next scheduled run (every 4 hours)
Pending	≥ 15 days	Send HR Manager escalation email; pause onboarding workflow

3.2 Processing Branches

Branch 1 — Pass

- Flask API confirms BG = Pass and risk policy approved
- Bot generates IT Ticket ID: IT-EMP###-YYYYMMDD
- Bot sends IT provisioning email to IT inbox via Flask /send-email
- Bot sends welcome email to new hire via Flask /send-email

- Bot writes all results to Google Sheets columns S-AA; Welcome_Email_Sent = Yes, IT_Email_Sent = Yes

Branch 2 — Fail

- Flask API returns BG = Fail
- Bot sends rejection notification to candidate via Flask /send-email
- Bot writes Bot_Status = Completed; Bot_Notes = "Background check failed. Rejection email sent."

Branch 3 — Pending < 15 days

- Flask API returns BG = Pending; days_waiting < 15
- Row is skipped; Bot_Status = Completed; Bot_Notes = "Background check pending. Retrying on the next scheduled run."
- Row is retried on the next scheduled run (every 4 hours)

Branch 4 — Pending ≥ 15 days

- Flask API returns BG = Pending; days_waiting ≥ 15
- Bot sends HR Manager escalation email via Flask /send-email
- Bot_Status = Completed; Bot_Notes = "Background check pending over 15 days. HR Manager escalated."

Branch 5 — Documents Missing

- Required documents field is empty or incomplete per Flask API response
- Bot sends HR notification email via Flask /send-email
- Bot_Status = Completed;

4. Google Sheets Write-Back

After processing each row, the bot writes all results back to Google Sheets using the Write Cell (Google Workspace) activity. A rowCounter variable tracks the current row number and increments in the Finally block of the Try-Catch wrapper to ensure subsequent rows always process correctly.

Col	Field	Source	Written When
I	Risk_Policy_Check	Flask API (PDF rules)	All rows with complete docs
J	Required_Documents	Flask API (PDF rules)	All rows with complete docs
K	Risk_Policy_Notes	Flask API (PDF rules)	All rows with complete docs

P	BG_Check_Status	Flask API	All rows with complete docs
Q	BG_Check_Date	Flask API	All rows with complete docs
R	BG_Days_Waiting	Flask API	All rows with complete docs
S	Account_Needed	Bot logic	Pass branch only
T	Account_UPN	Bot generated	Pass branch only
U	IT_Ticket_ID	Bot generated (IT-EMP###-YYYYMMDD)	Pass branch only
V	Equipment_Type	Flask API (PDF rules)	All rows with complete docs
W	Bot_Status	Bot logic	Every row processed
X	Bot_Last_Run	DateTime.Now	Every row processed
Y	Bot_Notes	Bot logic (branch-specific)	Every row processed
Z	Welcome_Email_Sent	Bot logic	Yes (Pass) / No (others)
AA	IT_Email_Sent	Bot logic	Yes (Pass) / blank (others)

5. Component Design

The bot is composed of seven logical components. Each component corresponds to a discrete UiPath activity group within the workflow. The Try-Catch-Finally wrapper (COMP-007) encapsulates all per-row processing to ensure fault isolation.

ID	Component	Description	UiPath Activity
COMP-001	Google Sheets Reader	Reads all rows from the new hire tracking sheet; populates variables for each column; increments rowCounter	UiPath Read Range (Google Workspace)
COMP-002	Flask API Caller	Sends HTTP GET to Flask /background-check with employee_id, department, role; receives JSON with risk classification, BG status, and required documents	UiPath HTTP Request Activity
COMP-003	Routing Decision Engine	Evaluates BG status and days_waiting; routes to Pass, Fail, Pending-Skip, Pending-Escalate, or Docs-Missing branch	UiPath If / Switch Activity

COMP-004	IT Provisioning Emailer	Generates IT Ticket ID (IT-EMP###-YYYYMMDD); sends provisioning email to IT inbox via Flask /send-email	UiPath HTTP Request + String Builder
COMP-005	Welcome Emailer	Sends welcome email to new hire with onboarding details via Flask /send-email	UiPath HTTP Request Activity
COMP-006	Google Sheets Write-Back	Writes all processing results to columns I-AA using rowCounter to target the correct row	UiPath Write Cell (Google Workspace)
COMP-007	Try-Catch-Finally Wrapper	Wraps all per-row logic; catches exceptions; logs employee ID and error message; increments rowCounter in Finally block	UiPath Try Catch Activity

5.1 Workflow Execution Order

COMP-001 (Read Sheets) → For Each Row: COMP-007 wraps → COMP-002 (Flask API Call) → COMP-003 (Routing Decision) → COMP-004 (IT Email, Pass only) → COMP-005 (Welcome Email, Pass only) → COMP-006 (Write-Back) → rowCounter++ in Finally.

6. Integration Specifications

6.1 API Endpoints

Endpoint	Method	Request Payload	Response	Status Codes
Flask /background-check	GET	?employee_id=EMP001&department=Finance&role=Financial Analyst&api_key=demo-api-key-001	{ risk_level, bg_status, bg_days, required_docs, equipment_type }	200 OK / 500 Error
Flask /send-email	POST	{ to, subject, body }	{ message_id, status }	200 OK / 500 Error
Flask /inbox	GET	None	HTML page listing all received emails	200 OK
Google Sheets Read	OAuth GET	Sheet ID + Range	2D array of row values	200 OK / 403 Auth Error

Google Sheets Write Cell	OAuth PUT	Sheet ID + Cell ref + Value	Updated cell confirmation	200 OK / 403 Auth Error
-----------------------------	--------------	--------------------------------	------------------------------	----------------------------

6.2 Authentication

- Google Sheets: Connected via UiPath Studio Web's built-in Google Workspace connector. The developer links a personal or organisational Google account through UiPath's OAuth consent flow. No service account or API key is configured manually. Access is scoped to the linked Google account's Drive and Sheets permissions.
- Flask API: API key required (demo-api-key-001); passed as URL query parameter `api_key`. HTTPS enforced by Render.
- Flask /inbox: Accessible via public Render URL; no login required for demo. Restrict in production.

6.3 Flask API Response Schema

The Flask `/api/onboard` endpoint returns a JSON object with the following fields:

- `risk_level`: string — HIGH/MEDIUM/STANDARD
- `status`: string — Pass / Fail / Pending
- `days_waiting`: integer — number of days since BG submission
- `required_documents`: array of strings — document names required for the role
- `equipment`: string — Laptop / Desktop / Mobile per role mapping
- `notes`: string — additional compliance notes from the PDF policy

7. Error Handling

7.1 Try-Catch Architecture

Every row's processing logic is wrapped in a Try-Catch-Finally block. Failures are logged with the employee ID and error message to the `Bot_Notes` column (Y) in Google Sheets. The `rowCounter` increments in Finally to ensure subsequent rows always process correctly, regardless of whether the current row succeeded or failed.

7.2 Exception Register

ID	Trigger	Bot Action	Human Action
EX-001	BG Check = Fail	Send rejection email; write exception notes to Sheets	HR reviews exception
EX-002	BG Pending $\geq$ 15 days	Send HR Manager escalation email; row paused	HR follows up with vendor

EX-003	Documents incomplete	Send HR notification; skip provisioning; write to Sheets	HR contacts candidate
EX-004	Flask API unavailable	Try-Catch logs error; row skipped; retried next run	IT checks Render deployment
EX-005	Sheets write fails	Try-Catch logs error; continues to next row	IT reviews Sheets connection

8. Design Decisions and Trade-offs

Decision	Challenge Encountered	Why This Approach	Trade-off Accepted
UiPath Studio Web over Power Automate	Challenge: Power Automate's HTTP connector required premium licensing for custom REST API calls. Free tier couldn't call our Flask background check API without workarounds.	Why UiPath: UiPath Studio Web offers native HTTP Request activity with full JSON body support, Google Sheets connector with OAuth (already authenticated via bearcatrpademo@gmail.com), and a fully free tier with no connector restrictions.	Trade-off: Studio Web has no code/script editor — every logic step must be a visual activity. This made dynamic cell addressing ("I" + rowCounter) require a dedicated Assign Variable pattern instead of a simple loop index.
Flask API as single combined endpoint	Challenge: Originally the background check API and Risk Policy classification were separate concerns. HR was manually copying PDF rules into Google Sheets — defeating the purpose of automation and creating a single-source-of-truth problem.	Why combined: By encoding RCP-HR-2025-001 rules directly in the Flask API, a single GET /background-check call now returns both the BG check result AND the risk classification derived from the PDF. This satisfies the two-data-source requirement (Spreadsheet + PDF) and eliminates manual HR data entry for risk columns.	Trade-off: Render's free tier hibernates after 15 minutes of inactivity. The first request after hibernation takes 30-60 seconds. Mitigation: keep the /inbox page open in the browser before running the bot, which keeps Render alive via the 10-second auto-refresh.
Built-in Flask email inbox over Mailtrap SMTP	Challenge: Mailtrap free tier has a 50-email/month limit and a rate limit of 1 email per second. With 6 employees × 3 email types = 18 emails per run, and repeated testing, the quota was exhausted within hours.	Why built-in inbox: By adding a /send-email POST endpoint and /inbox web UI to the existing Flask API, we eliminated all third-party email dependencies. The inbox displays emails in real-time with colour-coded types (Welcome, Rejection, IT Provisioning, HR Notification), auto-refreshes every 10 seconds, and has zero rate limits. Demo-day presentation is actually better. The manager can watch emails appear live at bearcat-rpa.onrender.com/inbox.	Trade-off: A Render restart clears the inbox. Acceptable for demo scope; in production, emails would use a persistent SMTP service (SendGrid, AWS SES) with proper delivery guarantees.

IT email notification over ServiceNow REST API	Challenge: ServiceNow Developer Instance (dev394946) repeatedly timed out with TaskCanceledException. The instance hibernates unpredictably, and even after manually waking it via developer.servicenow.com, HTTP Requests from UiPath still failed. The snowObject.result.number Deserialize step returned NullReferenceException because the response was empty.	Why IT email: A structured IT provisioning email sent to the Flask inbox provides identical information to a ServiceNow ticket: Employee ID, name, department, role, risk level, equipment requirements, AD account UPN, and a bot-generated IT Ticket ID (format: IT-EMP###-YYYYMMDD). The ticket ID is included in the email subject and body, making it traceable. This approach is 100% reliable with zero external dependencies.	Trade-off: IT_Ticket_ID is bot-generated rather than system-assigned. In production, we will link to the inner network platform to do cross department communication. Usually the account should be set up by the IT department because of risk control.
Google Sheets over Excel + SharePoint	Challenge: UiPath Studio Web's Personal Account tier restricts write-back to Excel files. The Write Range activity was greyed out. Attempts to use the Excel connector returned permission errors because SharePoint write access requires an organisational Microsoft 365 account.	Why Google Sheets: The Google Workspace connector in UiPath supports full read and write via OAuth, and the bearcatrpademo@gmail.com account was authenticated without any enterprise licensing. The For Each Row in Spreadsheet activity reads data, and Write Cell (Google Workspace) writes individual cells using dynamic addresses ("I" + rowCounter.ToString). Real-time updates are immediately visible in the browser — valuable for demo purposes.	Trade-off: Google Sheets API has a write rate limit on free accounts. To avoid hitting it, a 10-second Delay was added after each email HTTP Request. This adds ~60 seconds to a full 6-row run but prevents the "Too many requests" error that caused earlier test failures.

9. Test Cases

Demo link: [NewHireTracking_Bearcat - Google Sheets](#)

The following test cases validate end-to-end bot behaviour across all five processing branches and key exception scenarios. Test results are verified by inspecting Google Sheets write-back values and Flask /inbox email contents after each test run.

TC ID	Test Case	Input Condition	Expected Output	Result
TC-001	Pass – Standard Role	BG=Pass, docs complete, standard role	IT email sent; Welcome email sent; Bot_Status=Completed	Pass
TC-002	Pass – Finance Role	BG=Pass, docs complete, Finance role	IT email sent; Welcome email sent; Risk_Policy_Check=HIGH	Pass

TC-003	Fail – BG Check	BG=Fail	Rejection email sent; Bot_Status=Completed; no IT provisioning	Pass
TC-004	Pending – Under 15 days	BG=Pending, days_waiting=8	Row skipped; Bot_Status=Completed; no email sent	Pass
TC-005	Pending – Over 15 days	BG=Pending, days_waiting=25	HR Manager escalation email sent; Bot_Status=Completed	Pass
TC-006	Missing Documents	Doc_Transcript_Received=No	HR notification sent; no provisioning; Bot_Status=Completed	Pass
TC-008	Flask API Unavailable	Render endpoint returns error	Try-Catch catches error; row skipped; error logged to Bot_Notes	Pass
TC-009	Sheets Write Fails	Google Sheets API error	Try-Catch catches error; continues to next row	Pass
TC-010	Multiple Rows	Mixed statuses across rows	Each row processed independently; Bot writes correct result per row	Pass

10. Assumptions and Dependencies

ID	Assumption
A-002	Flask API is deployed on Render and the /inbox endpoint is kept open to prevent the free-tier sleep timeout
A-003	The Risk Control Policy PDF (RCP-HR-2025-001 v2.2) is embedded in the Flask API and will not change within the first 12 months of deployment
A-004	Background check status is simulated by the Flask API for development and demo; production migration requires live vendor API credentials
A-005	Google Sheets new hire tracking file uses a consistent column schema; columns I–AA are reserved for bot write-back

10.1 Known Constraints

- Render free tier: Flask API sleeps after 15 minutes of inactivity — keep /inbox open or upgrade to paid tier for production

- UiPath Studio Web: Visual-only configuration; no inline code editor — complex logic must be expressed through activity combinations
- In-memory email inbox: All emails lost on Render restart — acceptable for demo; production requires persistent storage or SMTP

11. Security and Data Privacy

Area	Control
Flask API Security	Deployed on Render with HTTPS; api_key=demo-api-key-001; production should add API key or token
Email Inbox	In-memory storage on Render; emails are visible to anyone with the /inbox URL; restrict access in production
Employee PII	Names, IDs, and document data processed in-memory only; no PII written to external logs beyond Google Sheets
Data Retention	Google Sheets log retained per HR data retention policy; Flask in-memory emails cleared on Render restart
Audit Trail	All bot actions written to Google Sheets columns W–AA; provides a complete, timestamped audit trail per hire

12. Next Steps

- Complete test case execution against the Google Sheets tracking file and document bot write-back results for all branches
- Prepare and rehearse the live bot demonstration for the final presentation
- Moving to production: replace the simulated Flask background check with a live vendor API and upgrade Render to a paid tier for uptime reliability